

Software Architecture

Scalability

10

Contents

- Scalability: Workloads and Resources
- Scale up or Scale out?
- Scaling Dimensions: Number of Clients (Workload), Input Size, State Size, Number of Dependencies
- Location Transparency: Directory and Dependency Injection
- Scalability Patterns: Scatter/Gather, Master/Worker, Load Balancing and Sharding

Will it scale?

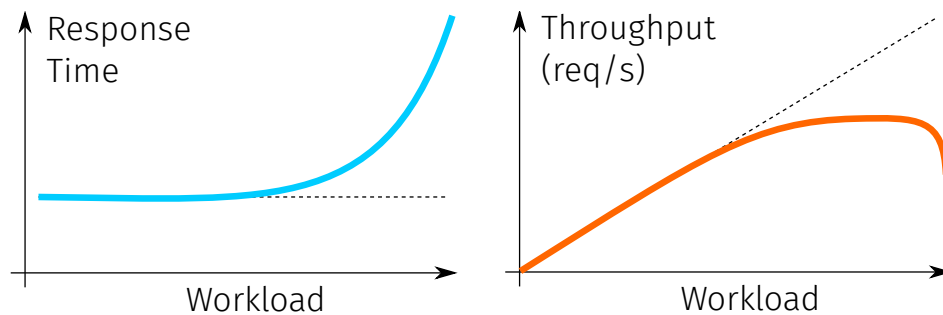
Scalability: this is the first of the quality attributes beyond raw performance that have to do with our system being in production at runtime. The question is: how do we scale the architecture?

When you hear about scalability, the term is often tossed around in discussions, scalability by itself doesn't mean anything. You have to be a bit more precise. You have to be able to put it into context. In which dimension do you want to scale your architecture? You can then produce a completely different design if your goal is to scale the number of concurrent clients that you want to support or you want to scale your system to be able to produce or consume a large amount of data. Or you want to be able to scale your architecture to be deployed over millions of different devices.

First we will define what is scalability and what are the challenges involved into designing systems that can scale. And then we will see what kind of solutions, what kind of patterns we can introduce in our architecture so that we can claim that our system is going to scale.

We start by defining scalability in terms of workload, or scalability because we want our system to fully use a large number of resources. We will see the difference between scalability techniques that involve decentralization or scaling up towards larger and larger centralized systems. What are the basic mechanisms at the foundation of architectures that can scale? We need to be able to enable the dynamic discovery of components, their identity and location: who is running where. And then we will look at more detailed scalability patterns: master/worker, load balancing and sharding.

Scalability and Workload



Workload = traffic, number of clients or their number of concurrent requests

Ideal system: response time not affected by the workload;
throughput grows proportionally to the workload

Real systems will show this behavior up to their capacity limit

To observe the scalability of our system, we need to measure first its performance. The performance can be measured in terms of latency with the response time. To perform a certain amount of work, the system takes some time. We can also look at performance in terms of throughput: how much work over time can our system perform?

In these charts we will see that the performance of the system is a function of the workload of the system. The busier your system is, the longer it will take to process the work. We start from a situation in which there is no impact of the workload on the response time. As the workload increases, the response time stays flat. It always takes the same amount of time no matter how busy the system is. Also we can see that the throughput grows together with the workload. The workload can also be defined in terms of requests per second. We can observe that there is a growing number of clients sending traffic to our system but the system can absorb the traffic.

The more work you send, the more the system gets busy. But since there is no impact on the response time, we can say that it scales and it will do so until it reaches its capacity limit. That's where it stops scaling with this behavior. And we say that it has reached a saturation point. The system is saturated because even if you keep increasing the workload (as shown with the dotted line) and send more and more requests, the system cannot cope with them. We recognize the saturation phase in which the throughput stays flat and the response time starts to increase.

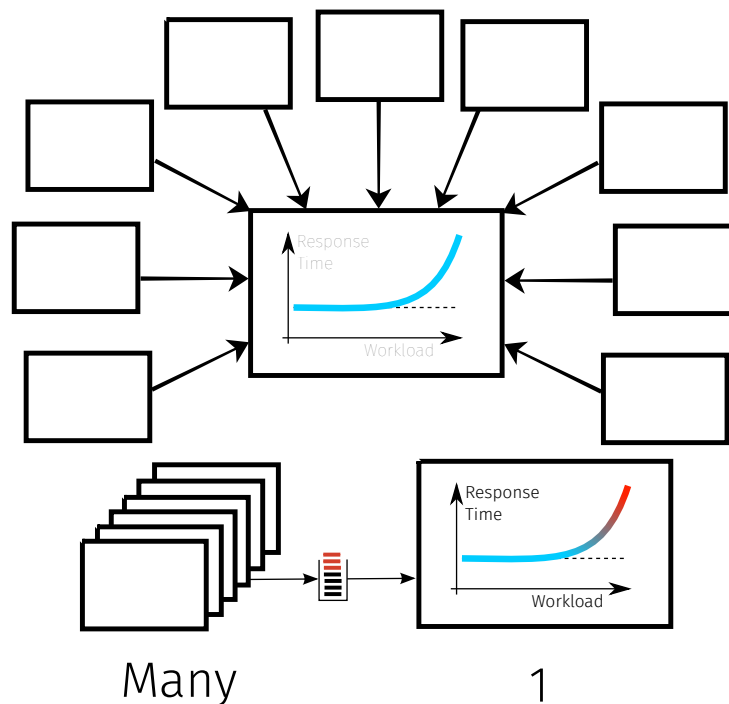
Let's not confuse the throughput with the response time. When the system scales, the response time remains flat and the throughput increases proportionally to the workload. When the system saturates, the throughput flattens and the response time starts to increase. This can be intuitive to understand: imagine there is a queue of request somewhere. Why would requests stay longer and longer in the queue? because the processing capacity is limited and only so many requests can be processed per second.

If you further increase the workload, you go from saturation to overload. And here we see that the system is completely overwhelmed. As a consequence, its behavior becomes uncontrolled, its queues get clogged, the throughput goes down and the response time goes beyond what the clients will accept. That's when you start to see disconnections. Clients start to get timeouts. Clients will react by resending their requests, further growing the length of your queue and overall the system becomes unstable.

To summarize: the workload your system is subject to can change over time. Depending on the level of the traffic, the number of clients, the number of concurrent requests from the clients, we can observe the performance of the system in relationship to the workload. Linear scalability occurs when the response time is not affected by the workload and the throughput grows proportionally. The more work you give, the more the system will be busy, but the system can keep up. Infinite scalability does not exist, as every real system has a limited capacity. The linear scalability behavior will occur only for a certain workload range. There will be a point in which there is just too much work. There is too much traffic and this system has reached its saturation point and if the workload keeps growing it becomes overloaded.

So this is true for every scalable architecture: there is always going to be a capacity limit. The question is how do you find the limit and how do you push the limit a little bit higher.

Scalability and Workload: Centralized



Where do these curves come from? There is a growing number of clients – this is where the work comes from – which call the component in the middle. In your architecture model you just draw two boxes, but then you know that one of the boxes will be instantiated many times, deployed all over the world and there will be many of those at runtime that depend on this central component to perform some work and they have some expectations about its performance.

If we look at this particular scenario, then we can say that the limit to the scalability that we have comes from the fact that this is a centralized architecture, in which there is one element that has to work for servicing requests coming from many elements. There is a fundamental imbalance: we have one element in the center working for all the others, which becomes increasingly busy the more clients you attach to it.

We can also observe its behavior by introducing a message queue connector. When the system is working within its capacity, it can consume the messages produced by the clients without them stacking up in the queue. In this producer/consumer scenario, the producer and consumer work at the same rate: every message to be processed will take the same time as we can see from the response time.

However, if we add more clients, there is more work to do: there will be a point in which the system is falling behind and the queue doesn't get emptied anymore. In the worst case scenario, also the queue gets filled up beyond its capacity and it has to start preventing clients from adding more messages.

In a centralized architecture the workload of a single component comes from a variable number of clients. Their number can grow or change dynamically. There is only one component that is going to service them. We can say that this component becomes the bottleneck: if this component is not fast enough, the architecture will not scale beyond the capacity of its centralized components.

How to scale?

1. Work faster
2. Work less
3. Work later
4. Get help



The question that I would like to ask is: How would you solve that scenario so that we can avoid completely filling up the queue? Clustering over many servers. Distribute the workload on multiple machines: good ideas. So let's spread out the load over multiple servers. If one server is busy, we find another one.

Another interesting idea is to put a time limit on the messages, so once we enter a message in the queue, we expect that this message will be processed within a certain time window, and if it takes too long, we drop expired messages. We can empty the queue this way without doing any work.

Another way is to make the server more powerful and in general to balance the processing power between producers and consumers. The queue helps to even out temporary unbalances, but overall you need to make sure the queue length does not grow unbounded.

Caching is also a great idea which helps to scale. This will basically make it unnecessary to send a message into the queue, since the client already knows the result. Avoid to recompute already known results. Invest into extra storage for the cache so that you save CPU cycles.

The problem of scaling comes from having to deal with a workload peak. Such peaks could be predictable or they can be totally unpredictable. Once your architecture has to absorb such a peak, you have to figure out how you design it by taking advance of your suggestions.

First, a very abstract and very simple principle that can help you is: Whatever you're trying to do, just do it faster. If you do it faster, it takes less time. That means that in the same time you can process more requests. This is where your algorithm optimization skills come in to play. Reducing the complexity of algorithm makes your system more efficient. If you cannot squeeze the algorithmic complexity any more, you can throw money at the problem: if you can afford it, you can buy a faster computer. Or you just wait a couple of years and then Moore's law is going to come to your rescue. Eventually things that used to be impossible or too expensive, they become affordable and easy to

do, thanks to faster and cheaper hardware.

The second idea is also simple: instead of performing the whole computation, we're going to compute an approximate result. This should also take less time, the result may not be as good, but at least clients do not timeout.

Another variation is not to return the freshest result, but to add a cache. Caching helps to reduce the workload because you remember historical results and then you look them up before sending or processing an incoming request.

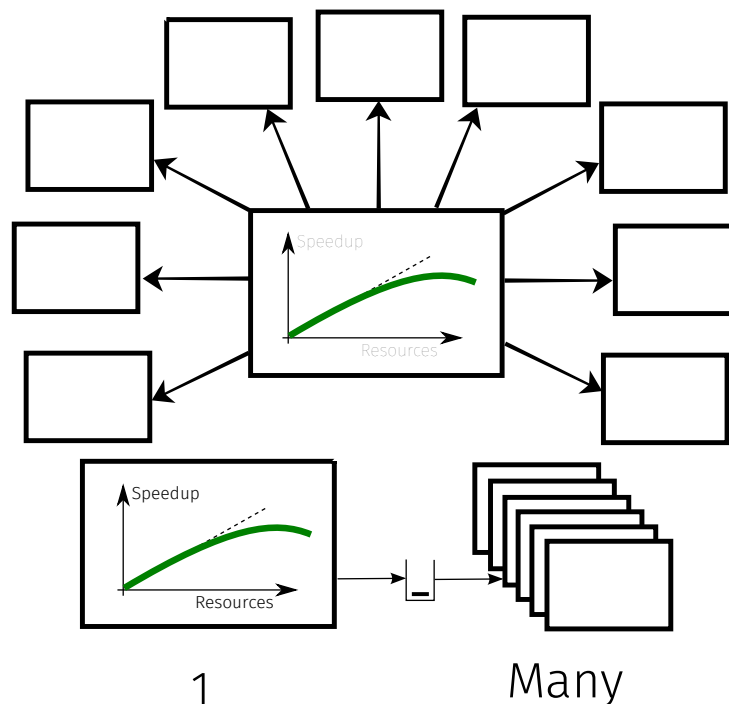
Or you can always push back and say I'm not going to do this for you. You do it yourself. And this goes even beyond caching. In this case we are actually offloading the work: we are not doing this here because I have limited capacity and you do it yourself that you have plenty of processing power. If you think about how powerful the latest smartphones are, you might as well offload the work on so-called edge devices. Parts of the components that used to require a powerful server to execute them, get to run elsewhere: this way you will make your server in the centre more scalable because the work is done on the mobile devices at the edge.

If you are in an emergency situation, sometimes you can attempt to control which features are you going to offer while you disable unimportant features. So you partially degrade the user experience, but you can support a much larger number of clients. Users will understand: you are undergoing a huge load at the moment and here is a very simple version of the application that still covers 80% of the use cases, but we have cut off the expensive ones. Come back later after the peak is past and we will reactivate them.

Another technique that you can use if you know that the peak is short lived is to buffer the work and then empty the queue after the peak is gone. If you survive the worst, you can catch up later. You don't necessarily need to perform the work right now if you don't have such real time constraints. If there is no deadline, you can absorb the peak when you have a time in which you're not so busy.

If all of these ideas are not enough. There is still another one which is also what you suggested right away: If you cannot keep up by yourself – there is always a limit on how much you can do – you will need to ask your friends for some help. So in this case, what you want is to be able to create multiple servers, add multiple cores, start additional parallel threads. Will introducing parallelism into your architecture help you to scale? Is your software going to benefit by adding more computational resources?

Scalability and Resources: Decentralized



Let's look at how parallelism affects scalability. In this scenario, when you want to spread out the work on a cluster, you reverse the relationship between the clients and the central server. Here we have actually one client that now has to spread out his work across multiple servers. So don't be confused by the location of the boxes. In the previous picture, the clients were outside sending work to the server in the center that was getting overloaded.

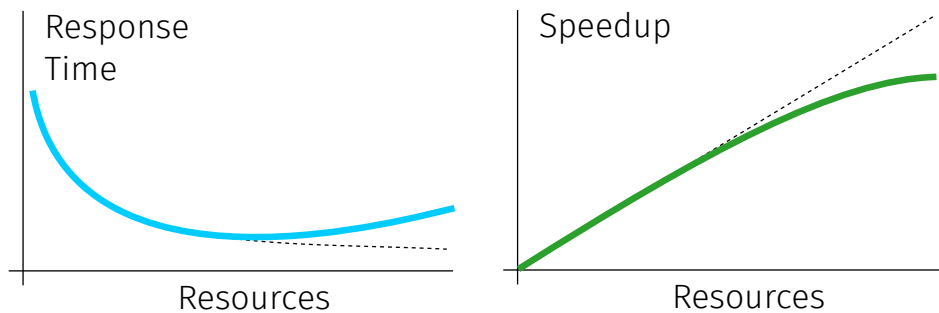
With a decentralized architecture we are inverting these roles, we have one client in the center which instead of talking to only one server, it is actually going to send the work across a large set of servers. The boxes on the outside represent multiple copies of the server which can receive the work.

If we look at it from the queue point of view, we are in a situation in which the client is overloading the server. The queue is getting full. There is too much work. So if you want to scale beyond the capacity of one consumer, you need to add more copies of the server. Working all together, they will be able to work in parallel and consume the queue and bring back the system in balance.

This is what we call a system in which the bottleneck has been solved by replicating the server components consuming the work to do from the queue. Even if you have a single client that is producing too much work for one server, you can absorb it and handle it in a scalable way by adding more and more resources to process the work.

Here we don't see performance as a function of the workload. Here we can try to study how does the performance behave in terms of the amount of allocated resources? How many copies of the servers we need to process the work? We still can measure the response time, but the X axis has changed. More precisely, we fix the workload. If we had more resources, if instead of just one server we add a whole cluster, or if we go from a cluster to a data center, or if we go from a data center to multiple regions all over the world: how is the increase in resources going to affect the performance?

Scalability and Resources



For the same workload, will performance improve by adding more resources?

Ideal system: linear speedup with infinite resources

Real systems will only benefit up to a limited amount of resources

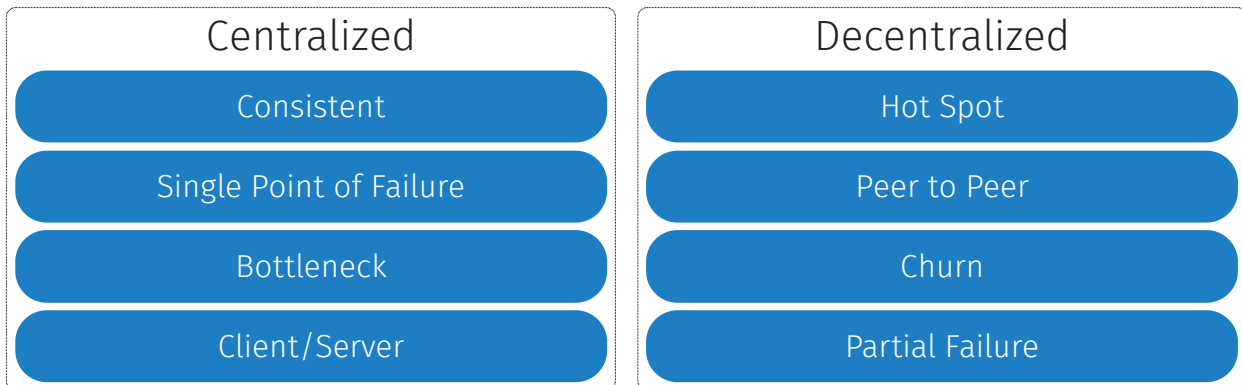
We have a scalable architecture if the response time decreases when we increase the resources available to process the given workload. With an ideal system, we will achieve linear speed up. The speedup (the performance relative to a centralized solution) improves linearly. It scales as long as the speed up grows linearly when you increase the resources.

For example, if you start from one server and then you double the capacity of the system, you now have two servers. With a scalable architecture you would expect the response time to be half of the previous configuration. And if you double the resources again, then you expect the response time to four times lower than the original one.

Can you keep doing this with an arbitrary amount of resources? Can you always double the amount of servers, double the number of cores with a processor and expect the performance to improve just like that? Also in this case, there is a limit after which actually you don't get any benefit. Sometimes it even gets worse: you buy one more server and the performance no longer improves.

There is a point in which the speedup doesn't grow linearly, but it actually starts to become flat. This is another limit of the scalability in terms of resources, with a fixed workload. Sometimes this is also called the law of diminishing returns. So if you double, if you go from one to two CPU cores. You have a big performance improvement in absolute terms. But then if you double again, for example, go from 20 to 40 resources, the improvement is not as big anymore. There will be a point in which it becomes too expensive to add even more resources for the very small improvement that you actually get.

Centralized or Decentralized?



Since we have seen these two sides of scalability with a variable workload in a centralized system or with a decentralized system with an increasing amount of resources, how are you going to design your architecture? Do you choose a centralized or a decentralized one?

Can you connect these two concepts with their implications?

In a centralized architecture some elements become a scalability bottleneck. Compared to other elements with a bigger capacity, the bottleneck is the weakest link along your chain where the performance is lost: the first element whose capacity gets fully utilized.

Similar to a bottleneck, a hot spot is the element most affected by a workload peak. Even if you have a decentralized system, when you have a hot spot, one element reaches its capacity limit sooner than the others and you need to devote more and more resources to it.

Client/Server in its simplest form implies a centralized architecture when you have multiple clients that share the same server. But you can also have a server architecture that is decentralized, with multiple replicas of the server. I would like to use the term to refer to an active client sending work to a passive server.

This should be seen as the opposite to a peer to peer architecture in which both client and server act as client and server at the same time. So this means that you have elements in your architecture which can both send and receive work to and from others, which is mostly found in decentralized architectures. Still, also large peer to peer systems will use so-called "super peers" in order to scale to run on a large number of resources.

If you want to have consistency in your state, obtaining it with a centralized architecture is much easier. Basically, if you have only one copy in one place, then by definition this is equal to itself, so this is easy to keep consistent. As soon as you introduce decentralization, e.g., with multiple replicas, you will copy the data in different places. You can

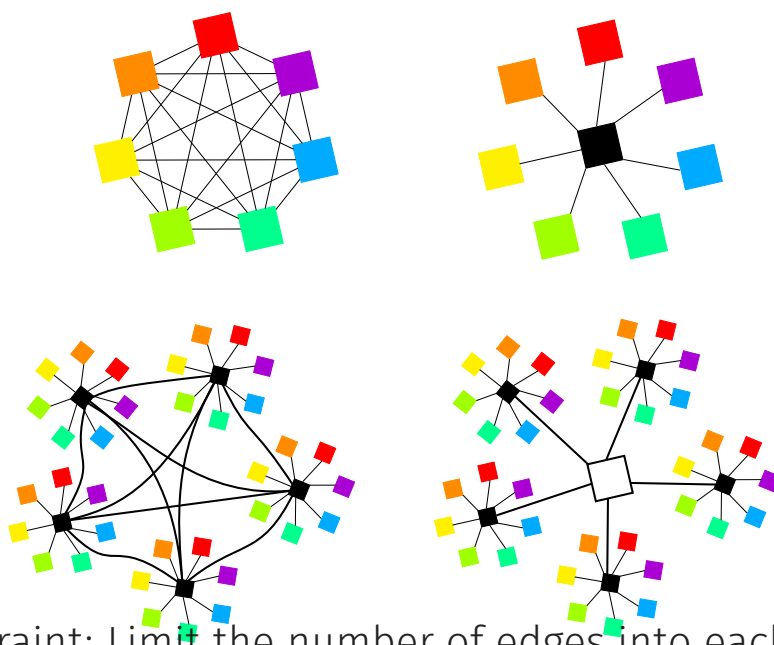
still try to keep it consistent, but it will be much more expensive to do so. So when you need to deliver consistency as a requirement, then you tend to pick a centralized solution. When dealing with scalability requirements, you may need to introduce replication within a decentralized architecture. In this case, what is the impact on the consistency of the data?

The decision also impacts possible failure modes. We will talk about failures more in detail when we go into the availability and reliability discussion. But clearly if you have a centralized architecture you have one element which becomes the single point of failure. Also with a centralized architecture, either everything works or nothing works.

Partial failure instead happens with a decentralized architecture: to have a partial failure, it means that you can split the system into more than one element. Some of those elements fail while others don't. This is a challenging situation to be in, especially when you have to recover your system, because while only some of the elements are down, so the rest of the system continues to work, when you restart them they will need to be brought into a consistent state with the others.

Churn is also typical of decentralization: the more elements are part of your system, the higher the probability that some of them will not be available. As your architecture scales to run across a very large number of resources, be prepared to deal with a continuous stream of disappearing, reappearing and lost devices, which will need to be continuously replaced with new ones just to keep the system running.

Scalability at Scale



Constraint: Limit the number of edges into each node

Let's turn the design of a scalable architecture into a graph problem. No matter whether you choose a centralized or decentralized design, you will find a limit. It's typically networking problem: the amount of connections that we can establish into the various elements of our system is limited.

If you look at these graphs. What is the maximum number of connections N at each node? In the first case, there is no center: every element is connected to every other one. If you look at the second graph, this number is no longer a constant value for every element. What is the limit here? Most of the elements have only one connection, but in the center, here comes the bottleneck. How many connections does the black node can support? Maybe it will be thousands, a high number, but still limited. This intrinsically limits how many components you can attach to the center one. So if you go for this solution – with the centralized architecture – there will be a physical limit to the number of network connections that you can have towards all the clients that you put around your server. The limit may depend on the hardware, operating system configuration, networking protocols, but there will be a limit.

What if you choose a decentralized solution? Does this limit disappear? Decentralized means that you now remove the server from the picture and go peer to peer. This means that all the elements in your architecture will potentially connect to everyone else. What is the maximum number of connections that you can have now in this fully connected graph? Exactly, so the number is still N . Getting rid of the server doesn't really solve your problem, because now every node behaves like the original server and every node is limited like the central server used to be.

How to scale beyond this limit? You have to remove the assumption of establishing a fully connected graph. You have to work on your graph topology under the constraint that there is a limit on how many connections each node may have.

You can take this idea of having a central element. And you can do it recursively (or

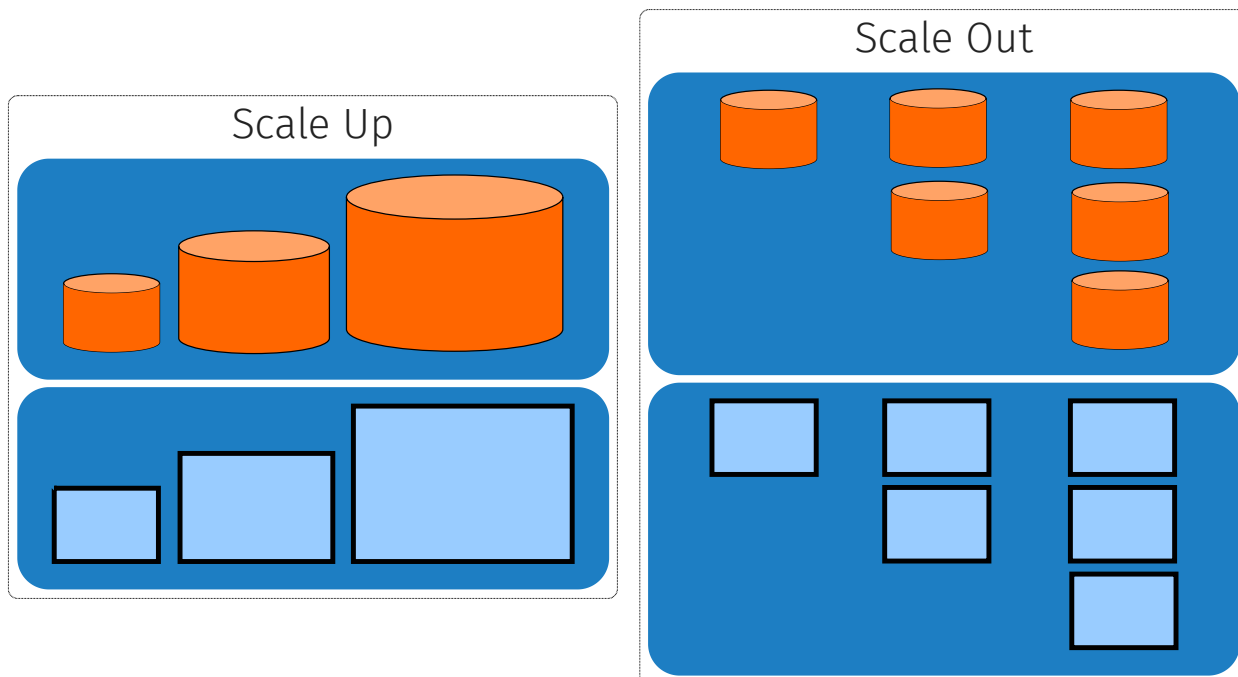
hierarchically): you can see here we have the black central element, which is connected itself to the white central element of the central elements. So we have a tree of two levels. We call this hierarchical architecture. Every element of the system can find a route through the tree to talk with all the other ones. There will be a lot of traffic towards the root of the tree: the next limiting factor will not be the number of connections of each node but the bandwidth of each edges.

One way to scale is to remove the root and have a hybrid topology, which sometimes is called a peer to peer system with super peers. Peers that are stable and perform better, and therefore we attach to them a local neighborhood of clients. Then the super peer themselves establish an overlay network, which can deliver more bandwidth than having to go through a single root node. We split the root and allow the black elements to connect directly to ourselves.

There is a third approach not shown in the pictures which is the epidemic or gossip based design. Here we have a random subset of the elements which connect in a way to keep the graph fully connected while minimizing the number of hops between the nodes. With high churn it's not easy to keep the tree topology in place and a random graph is more resilient, although more difficult to visualize.

I hope this idea helps you to design something that can grow beyond these fundamental limits. Removing the centralized element is not enough, since the connectivity bottleneck is still there for every element of the architecture. Instead, you have to be clever with the interconnection structure that you establish and learn from the experience of peer to peer networks.

Scale Up or Scale Out?



Let's look now at how to scale the nodes themselves. How can we increase the storage capacity? How can we get more powerful processing?

There are two ways, in which either we grow the size of the center or the architecture becomes more and more decentralized.

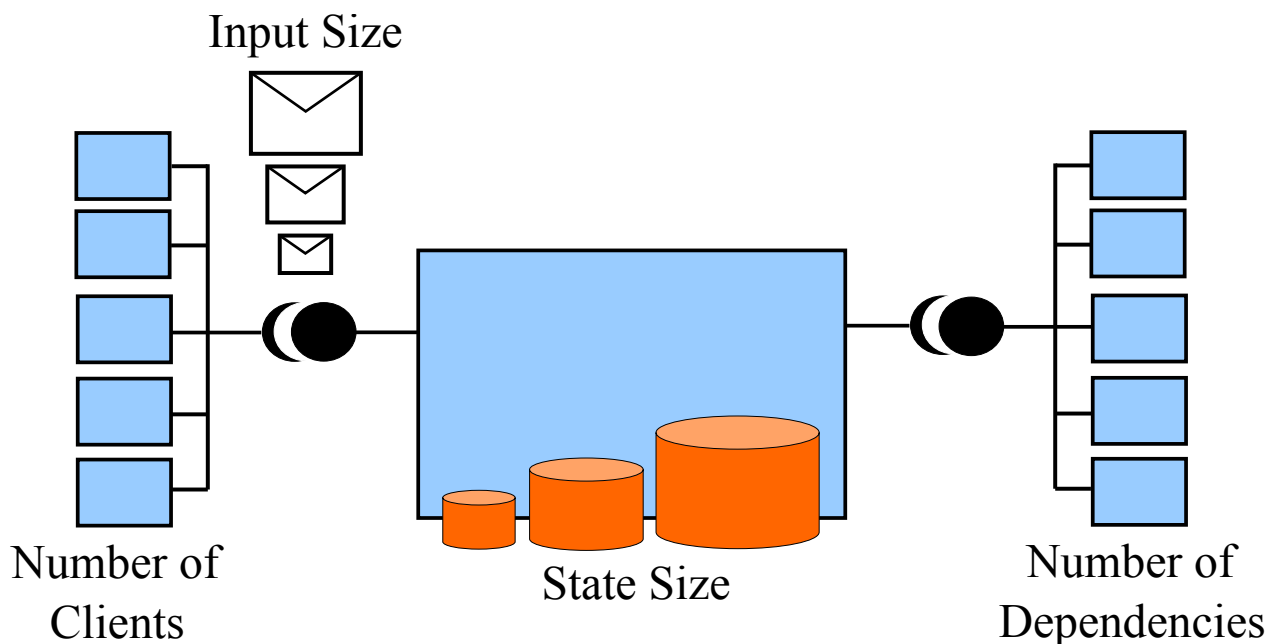
Scaling up means that we start from something small. As it grows, we keep it centralized. We keep it within the same element. We just make it bigger and bigger. For example, you buy your phone and you estimate: one terabyte of storage will be enough. What to do if it gets full? Buy a bigger phone, a phone with more space. The same holds for the memory and processor speed and cores. Is your system too slow, run it on a faster CPU with more cores. Also in this case, you reach a limit. The maximum available storage capacity is technologically limited, or its cost simply becomes too high.

Once you reach the limit of scaling up, you can still scale out if you need to grow further. Just take multiple copies of the same thing and place them side by side. Even if the size of the individual disk is limited, we just use two or more discs instead of one. As a consequence, we have twice as much storage. One processor is not fast enough: if you can, parallelize your code to take advantage of multiple cores.

Which is more expensive? Scaling up or out? While the price of adding capacity to an existing node may become increasingly expensive, the price of adding more nodes grows linearly with the number of copies. Still, there is a coordination overhead. When you add more disks, you need to invest in the RAID controller. When you add more servers, you need to invest in a good load balancer.

Scaling up and out are not alternative, they are often used in combination. First you try to scale up. Once it becomes too expensive, you can still scale out.

Scaling Dimensions



So far we have been trying to define the challenges of scalability. Let's summarize them with a map to understand the different solutions, because the problems that we face are not exactly the same.

We need to scale our system because its workload grows. The workload of the system is generated by the clients that concurrently send requests into our system. So the number of clients is dynamic and unpredictable. It really depends on how successful your application is, the more clients you will need to serve. Even if you have only one client, they may send you a growing amount of data (larger requests or more frequent requests). So these two factors affect the workload independently.

Another dimension that we will consider is how large is the amount of state that you need to manage. Some systems are stateless. These are easy to scale in terms of a large workload. In the extreme case you can start a server for each client. Stateful components are not so easy to scale. And special sharding techniques are used to grow the amount of state beyond the capacity of individual disks or storage elements.

While we discussed the case of one server under the load of a growing number of clients, we can look at the opposite perspective of one client which needs to work with many servers. Will this decentralized architecture scale in terms of the number of dependencies that this client can have? How to take full advantage of a growing amount of resources? How to share the load among a larger and larger pool of workers?

Scalability Patterns

Number of Clients (Workload)	Load Balancing
Input Size	Master/Worker
State Size	Sharding
Number of Dependencies	Directory Dependency Injection Scatter/Gather

If you want to scale in terms of the number of clients, if this is your challenge, you're facing a peak of workload. Then load balancing as a solution comes to mind. If you assume the request of each client to be independent from every other one, you can share the growing load by adding multiple servers and making sure that, on average, they're all busy in a similar way so you have a fair allocation of the work.

What if you don't have many clients, but you just have one client that is sending you a huge amount of work so the input size grows beyond your capacity? Is it possible to partition this work in independent units? You introduce the master worker architecture so you can still take advantage of a growing amount of resources.

If what you're trying to grow is the amount of storage, look into data sharding.

In general, all of these solutions require to scatter the work (or the data) and then gather the results back together when it's been processed. To know where you have to send it, you have to discover which workers are available, where they are and we will see that this can be done in two different ways with the directory or with the dependency injection patterns.

Directory

How to facilitate location transparency?

use a directory to find interface endpoints based on abstract descriptions

Clients avoid hard-coding knowledge about required interfaces as they **lookup their dependencies through a directory** which knows how and where to find them

Let's start from the foundations so that we can build a system that can dynamically grow and scale.

This foundation is about being able to discover where are the components and the resources that we can count on to process the work. Given this queue of incoming work, we want to be able to switch between different instances, replicas or alternative workers that execute the same component. To deliver the capacity we need to work at scale, we need to be able to switch between them so that we can send the work where there is capacity available.

If you will talk about scalability in terms of storage, in terms of the amount of state that you have to store, then the question is: where is this data going to be located?

If you go back to the lecture about continuous delivery pipelines and deployability, there we can also use discovery as a tool to configure the system to use the right dependencies along the pipeline. You have a system that is in production. It will have some precise dependencies that cannot be mixed with the dependencies that you use for testing. Typically this is a fundamental for what concerns the storage of the data of your system. The state of the system in production is critical, whereas the one that you use for testing should be separate. If it gets corrupted you can still regenerate it. While if you lose the production state, it's a major problem. You have to be able to configure this system so that when you start in a testing environment it will not use the same dependencies that you use in production.

If you think about different types of releases, when we do a blue/green release or we do these pilots, then you need a way for binding clients with different versions of your implementation. When you make a new release, you only want 10

In other words, when we talk about scalability, you have typically a homogeneous system in which the component implementations that you instantiate are all the same, you just have multiple replicas if you need more capacity. If you talk about the evolution of your system, then you have a chance to have a system that is not exactly homogeneous and you might have different versions and different traffic being routed to different versions of the implementation.

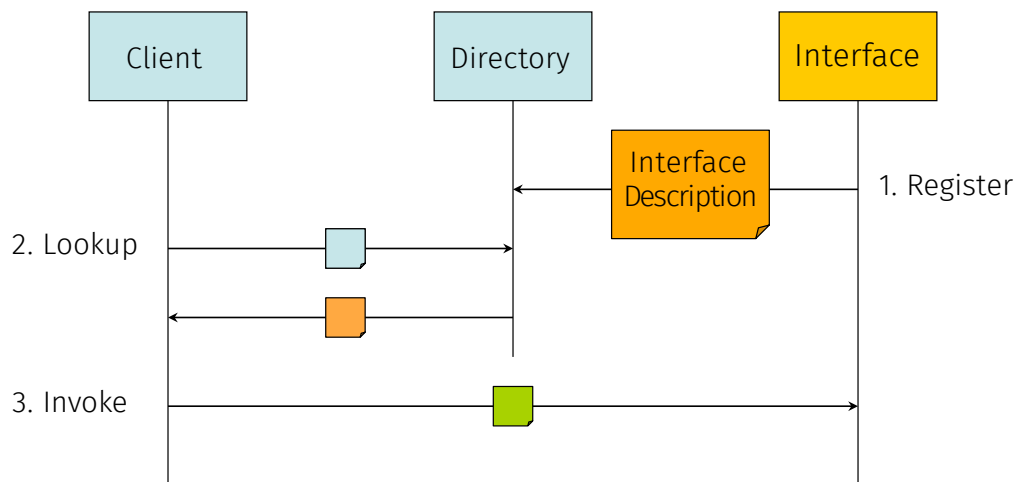
If you use discovery you're trying to make components independent from each other

regarding their location. Even if most components have dependencies, they should not care where they are: this is called location transparency. You can use a component without knowing where it is. Because when you need to talk to it, somebody will find it for you and will put you in contact. That's the job of the directory, a solution for many discovery problems.

If we want to facilitate location transparency, if we want to make it possible for a component to discover where is another component, so that for example it can send some work there to be processed in parallel, then you can use a directory to look up its location, to look up which component I should talk to based on some requirements, to perform some match making between what you need and what the component provides.

The goal of the directory is to know about what, where and who is available, which components are available and where they are and to perform the matchmaking between the components they depend on the components that need to be found to satisfy those dependencies. So this works because the clients do not need to know where the dependencies are and they can look up these dependencies through the directory at the latest possible moment.

Directory



Clients use the Directory to lookup published interfaces descriptions that will enable them to perform the actual invocation of the component they depend on

When a client is about to make a call, it needs to know where to find the component to be called. The directory can give its location. Here is how a directory works.

We start from the provider side. There is a component offering an interface with a certain description. This description is registered with the directory to announce its availability. Registration is where we say that this is who we are. This is what we can do, and this is where we are. So the directory remembers that.

Later, at a certain point in time, a client comes and it performs a look up. The client asks: I'm looking for a component that matches my requirements. The requirements come into the directory so that the directory goes over the set of registered interfaces, perform the matchmaking, and decide if there is a suitable interface that can satisfy your requirements.

If the matchmaking is successful, the lookup response will contain the location for the implementation of that interface.

Then the client can directly talk to the other component: the third step will be the actual invocation. This will be the actual message that goes between the client making the call and the server being called.

The idea of the directory is that clients are actively looking for suitable implementations of their dependencies. The directory knows where to find them and tells the client about their location.

Before we do the look up, we know what we need, but we don't know whether it exists nor where to find it. And after a successful lookup, we know it exists and where to find it. Then we can use it. This assumes that the directory has a successful matchmaking. It's also possible to try to look up something in that directory, and the directory doesn't find anything suitable, so a lookup may fail or the result of the lookup may no longer work.

The information that was registered may become obsolete. So even with a successful match, there is still a risk that it was based on obsolete information. Only after you do

the actual invocation you will know if everything has worked.

If you have a whole architecture with many components and with many dependencies among them, typically there will be one element of the architecture which plays the role of directory. It will be responsible for managing the location of the components and knowing which components are there and keeping track of where they are. This critical element will be used by all the clients to do their look ups. One important issue that we have is: we have a single directory for the whole system. As with every centralized design: is this directory going to become a single point of failure? If the directory doesn't answer the lookups, the clients are basically unable to perform their work because they don't find where to send their calls.

The directory may also become a performance bottleneck: if all the client lookups happen at the same time, how long does it take to get an answer? Before a client can do the actual work, they're waiting for the directory to provide the information needed to find where to send their invocation. Lookups need to happen as fast as possible. So in which way can we improve the design to deal with this issue?

We can apply some of the concepts that we discussed before to this concrete design problem. We have a slow, centralized directory. So we could have multiple directory instances and load balance each client to a different replica. The good thing about this replication is that the data of the directory doesn't change very often: we start the system, the interfaces are registered and then if the system is not too dynamic, we can just assume that the information in the directory is pretty static, so it's easy to make a replica.

We can try a radical approach, fully decentralized, so each client keeps his own directory. So what we're going to do is to deploy these two elements (clients and their directory) closely together so they run in the same host. The local directory lookup table can be synchronized through gossip, so each client will transfer all the changes to his neighbors.

Once an interface wants to register, it has to find out where at least one of these copies is in order to send information about its description there. Eventually this will be propagated around and reach the other clients. This is a nice idea if we want to remove completely the central element and have a copy of the directory running in each client. The downside would be probably that it takes time before the updates reach all the clients because of the huge redundancy that we introduce in the system where everybody has their own independent copy.

What if I ask you: do clients need to perform the look up every time? In this sequence diagram we see a look up before we do each invocation. Is this really necessary? If you want to avoid the cost of doing a look up before every invocation, we can use a cache in the client. This is also a copy of the directory content that gets populated on demand. After clients send a look up request to the central directory, they will store a copy of the result in the cache. If they need to look up the same interface twice, the second time they already have the result locally. So this is a way to scale the directory because you do not send so many requests to the directory, you only send them the first time. What if the cache becomes obsolete? This means that the invocation will fail because the client attempts to reach an obsolete endpoint. They are trying to reach a location that is no longer there. In this case, clients can look up again with the directory to refresh their cache. So this is an example of how we can scale the directory by caching the results.

You can generalize this idea and apply it anywhere you have a client retrieving information from a component, and this information remains valid for a certain amount of time. You can avoid resending the same request if you can recycle the results from previous invocations.

Let me ask you a different question now. We said that the directory helps the clients to locate the implementation of a certain interface, so the directory knows where to find

the components in the system, and the client uses the directory to find out where are the components. However, the directory itself is a component. The directory is a component which has a very simple interface that contains the register operation and the look up operation. The directory is a component which needs to be deployed and started on some container like any other component in your architecture. We can use the directory to locate all the other components; but how do we find out where is the directory going to be located?

The location of the directory has to be known by the clients in advance. In other words, the directory gives us location transparency for all the components in architecture apart from itself. Clients do not have to know where to find the components, but they have to know where to find the directory. Introducing a directory makes it cheap to move around components because you just have to update the directory and then the clients will dynamically retrieve this updated information, but if you want to move around the directory itself, then you have to probably reconfigure all rebuild or your clients depending on how strongly hardcoded is this information.

Another way to discover the directory's location is to use some networking tricks, such as using broadcasting. Clients will advertise their existence and the directory will be listening for such messages and perform a rendezvous. This would generate too much traffic for all components, but you can afford to do run this protocol to let each component discover the directory on startup. That can be another solution that avoids too hard coding the knowledge into the clients themselves.

What kind of information do you store in a directory? This is about domain modeling for interface description and discovery. For example, consider the domain name system (DNS). This is a very simple type of look up where you go from a symbolic name to a numeric IP address. Directories can also store a complete copy of the interface, or the actual interface is stored by the component itself and the directory just stores a reference to it. That depends on how easy you want to make it to update the directory: if you store a copy and you have to replace it every time you change it. If you store a reference then you just update your local copy and the directory will follow the reference to the latest version.

How complicated is the matchmaking performed by the directory? Given the syntax and semantics of the client dependency the directory can attempt to retrieve a compatible implementation. You can also have business constraints such as usage prices or rate limits.

Whatever metadata the directory collects, it is fundamental that it can be trusted. Directories are not only a performance and scalability bottleneck, not only a single point of failure, but also a security weakness. If you want to attack the system, you pollute its directory. You can call it a phishing attack or just an attack which exploits the trust the clients have in the lookup results from the directory. Clients will invoke whatever component the directory tells them to talk to. This requires strict validation of who is performing the registration. If you have an attacker, the attacker will register or update the address of an existing component and in the future clients will send all their messages to the attacker. To prevent this, while the look up can be relatively open, the registration has to be done only by authorized components.

If you run a directory for several years, the directory will contain a history of all your deployments of the system because every time you deploy the system, the components will register themselves and the directory will store their location. Every time you redeploy, maybe you change the location so the question is whether you want to keep historical record or you want to clean up the directory from time to time and have an expiration date associated with the registrations. If you register a component, the registration will remain valid for a certain time window, for example, a couple of months. After that time,

unless the component register itself again and this information will be purged from the directory. This avoids that it just accumulates there and it becomes obsolete. Obsolete information means that clients will fail their invocations due to the out of date lookup results.

In this simple design that we have shown here, there is no feedback: if the directory information is obsolete, the client invocation fails, but clients don't have a mechanism to inform the directory: "by the way, the location was incorrect, because the invocation failed. More sophisticated directories will either have a monitoring solution inside the directory that periodically checks whether the registered implementations are still where they claim to be, or will accept feedback from the clients and keep track of success/failure rates to estimate the availability of a given interface.

This was the first example of one of the basic building blocks of scalable architecture. The directory is an architectural pattern that delivers location independence, and every time you connect two components you will need to decide: How are these two components going to discover each other? How do they find out where they are so they can talk to each other?

We will see that directories are not the only way to solve this problem. With directories, the client is actively going to look up its dependencies. It's also possible to completely flip the relationship between the client and the directory. This is called dependency injection.

Dependency Injection

How to facilitate location transparency?

use a container which updates components with bindings to their dependencies

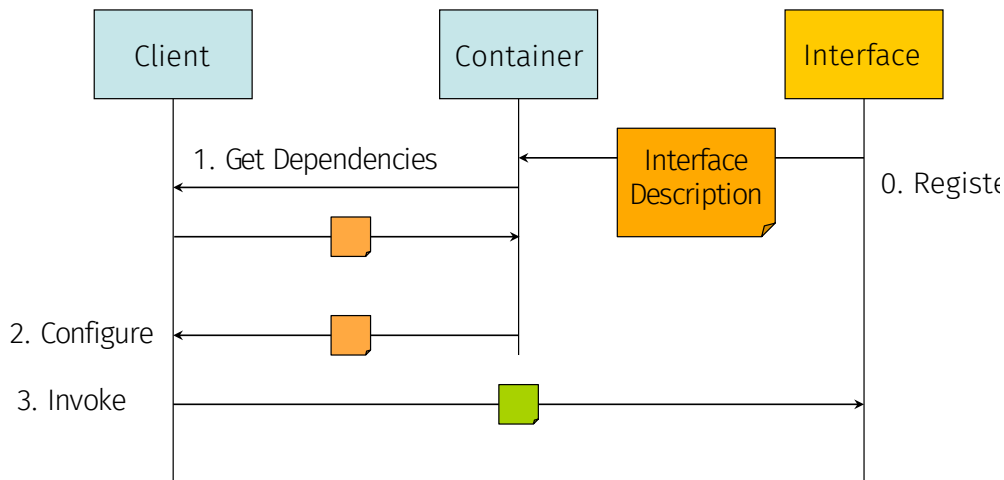
Clients avoid hard-coding knowledge about required interfaces as they expose a mechanism (setter or constructor) so that they can be configured with the necessary bindings

In the directory pattern, the client calls the directory to perform lookups. The interaction starts from the client, while the directory is passive. This relationship can be inverted. We can also use a directory to configure the dependencies of a passive client, which does not or is unable to call the directory. That's what the dependency injection is about.

With dependency injection, we assume there is a container in which components are deployed. The container is responsible to update its components with knowledge about where to find their dependencies.

In the same way as we had the directory, clients do not need to know where to find their dependencies. But they have to expose a mechanism so that they can be configured with the information they need to locate their dependencies. This way, the container can feed the information into them so that they can locate where are their dependencies without having to ask for them, and that's the most important difference.

Dependency Injection



As components are deployed in the container they are updated with bindings to the interfaces they require

So you can see how the interaction works. We still have the registration phase – the same as with the directory – where the interface advertises its location to the container.

When the component is deployed inside the container, the container would ask the component for its dependencies. The interaction starts from the container and the client is passively answering this request. The client will typically do so with some configuration file and there will be a way to get the metadata for this particular component. The container will inspect it. It will check that these dependencies can be satisfied based on the registrations that have occurred so far.

If the match making is successful, it will inform the component by configuring it so that when the component starts processing and needs to invoke a certain interface, it already knows where to find it.

These steps happen during the initialization of the container. As you start the system, the dependencies are injected and afterwards the components can operate normally and proceed to the invocation under the assumption that their dependencies have been satisfied and located.

As you can see the configuration part of the dependencies of the client is passive, so the client – the component that needs to be configured – is not doing anything by itself, it is just reacting to the container that injects the dependencies.

When you use a directory, you can wait until the latest possible moment before you need to know where to find your dependencies. So this is late binding, very dynamic while here we anticipate the binding at startup time. So with dependency injection you have a startup phase in which you configure the dependencies so that you don't have to do it later. This can save time during the actual invocation, because the invocation target is already bound.

If the location of some dependency changes between the startup time and the time of the call, you will need to reconfigure the clients. Whenever the registration changes, you have to see which clients are affected. Then you can still deal with it to avoid failed invocations due to obsolete registrations.

Dependency Injection

- Used to design architectures that follow the inversion of control principle:
 - “don't call us, we'll call you”, Hollywood Principle
- Components are passively configured (as opposed to actively looking up interfaces) to satisfy their dependencies:
 - Components should depend on required interfaces so that they are decoupled from the actual component implementations (which may be changed anytime)

Such inversion of the relationship between the directory and the component that is looking for dependencies is called “Hollywood principle”.

When you go to an audition, the usual answer at the end of the audition is that we need to decide whether you got the part: Please don't call us, we will call you (in case your audition was successful).

This clarifies the relationship between who has the information and who needs the information. We know the dependencies that you need. We will tell you how to satisfy them. You don't have to ask us, we'll do it ourselves.

With dependency injection components are passive. And they do not know how to look up for the interfaces that they need. They just sit there and wait to be configured. The idea is that you have components that know what their dependencies are and they are configured by an external entity so they can actually satisfy these dependencies.

Dependency Injection

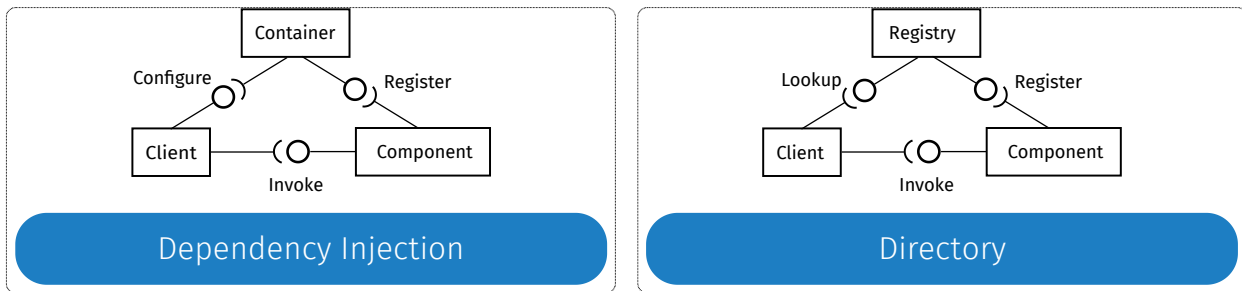
- Flexibility:
 - Systems are a loosely coupled collection of components that are externally connected and configured
 - Component bindings can be reconfigured at any time (multiple times)
- Testability:
 - Easy to switch components with mockups

The environment in which would deploy the component is in control on when and where these connections are established. This happens, of course when you start, but there can also happen later. In case you need to move some of the dependencies to a different location with the directory, you would reconfigure the directory. As soon as the client would try to call the old location, the call would fail. The client will look up again in the directory and be given the updated information. With the dependency injection you can prevent the failure, so you can just whenever you know that something has changed, you reconfigure the components with the updated information.

Another advantage of using dependencies injection is that it helps you to use the correct configuration along the build and continuous integration pipeline. If you deploy a component in a testing or staging environment, the container will configure the dependencies so that you can use for example mockups. Or components that are supposed to be used only during testing. When you deployed in production, the container that is configured as a production environment, it will use the correct dependencies.

Since the component is passive and the responsibility for setting up the right environment is all outside of the component, if for some reason the component forgets to make the look up with the right directory and relies hard coded information, this will be difficult to rectify without rebuilding the component (e.g., to avoid using testing dependencies when deployed in production) to avoid deploying the system with inconsistent dependencies. It can be easier to check that this doesn't happen by controlling the environment as opposed to each component.

Directory vs. Dependency Injection



To summarize the difference between directory and dependency injection, let's take a look at the logical perspective of how the three elements depend on each other. The component the client depends on is going to register with the container. And here we have the same: the component registers itself with the registry, which is a synonym for the directory. Also the invocation is the same on both sides, where the client invokes the component.

What is the only difference? It's a small one, but very important: the relationship between the client and the container/registry. With the directory, we have the client looking up the location of the component that wants to invoke in the registry, so the client is active while the registry is passive. With the dependency injection we have exactly opposite. The container is going to configure the component that is deployed inside the container setting the location about the component's dependencies. The container is active, while the client is passive.

Scatter/Gather

How to compose many equivalent interfaces?

Broadcast the same request and aggregate the replies

Send the same request message to all recipients, wait for all (or some) answers and aggregate them into a single reply message

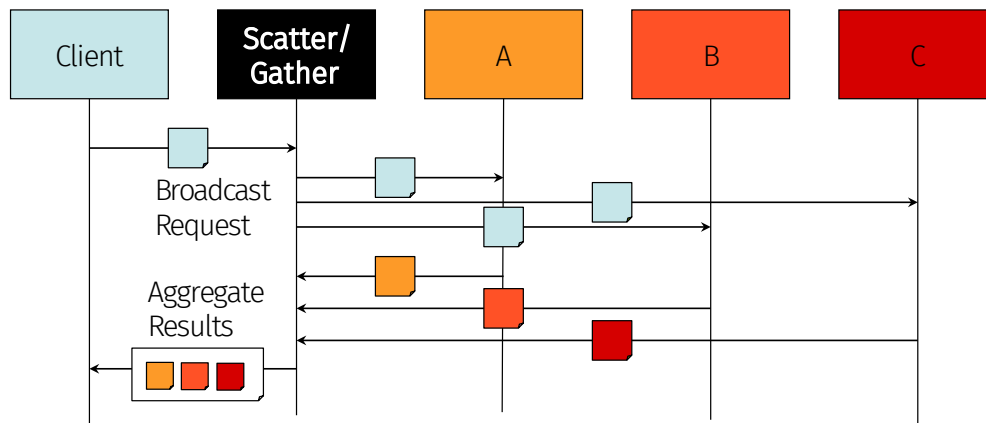
Scatter/Gather is a structure that we are going to use for understanding how scalability patterns such as load balancing, master/worker and sharding work.

Let's take it from a more abstract perspective first. The problem here is that we are in this decentralized design in which we have multiple equivalent interfaces that we can use. The question is: how do we make use of them as a whole, single unit? How do we compose them together so that from the client perspective they behave as a single component?

What we can do is, when we receive a request from the client, if we send this request to only one element, this would soon become a bottleneck when filling up its capacity, as a single element processing requests would be centralized. So we decompose the architecture by creating multiple copies of these bottleneck elements. Then we broadcast the request of the client to all of the copies, to all replicas. Once the replicas process the request we aggregate the replies. And we can send them back in one response message to the client.

How do we perform the request broadcast? And how to perform the response aggregation?

Scatter/Gather



Example:

- Contact N airlines simultaneously for price quotes
- Buy ticket from either airline if price ≤ 200 CHF
- Buy the cheapest ticket if price > 200 CHF
- Make the decision within 2 minutes

We have one request message coming in from the client. This request can be copied so we can send the same request to all the replicas. But then we may have different answers from each of them, both in terms of their content and their timing.

If we have a set of answers, how do we reduce the set of answer to only one? Because the client expects a single response.

Do we really need to wait for all of them? If our goal is to keep the latency under control, in order to produce a fast response, we could just send back to the client the response of the fastest replica.

One example: you need to travel somewhere. Different airlines receive the same input about your trip and respond with an offer describing a possible connection and its price. The rule for aggregating the results can be as follows: If you get a cheap price, you take the offer, there's no need to wait any longer. However, if the prices are a bit more expensive, you want to wait long enough to compare different answers. And then you will take the cheapest. However, you cannot wait forever, so if you found an answer within 2 minutes which is within your budget, you will buy it.

This gives you both a way to aggregate the results because you will take the cheapest together with a strategy to decide until when do you want to wait.

If one of the backends that you contact is overloaded, it will not answer within the deadline. It's better not having to wait forever for all replies, but to increase the chances that at least one replica will reply before the timeout.

Scatter/Gather

Which components should be involved?

- The recipients are kept hidden from the client
They can be dynamically discovered using subscriptions or directory registrations
- The recipients are known a priori by the client
The request includes a distribution list with the targeted client addresses

How do we know which replicas you want to invoke? There are two options to decide this. One is to keep the fact that you're scattering out the request hidden from the client, so the client thinks that it's talking to one component, but behind it you have multiple ones. Alternatively, the client is actually going to tell you who do you want to invoke.

This makes a very big difference in terms of how transparent your interface is. If you keep it hidden, it means that you are responsible to manage the replication of the implementation, the discovery of the back-end replicas, (you can use a directory behind the scenes) and scattering the client request and gathering the replies. The client doesn't need to know. Otherwise, you push all the directory management and all the knowledge about where to forward the request to the client. Whenever the client sends a request, the request will include the list of addresses of the components that the clients wants you to target.

Considering the airline example, it makes sense that the client can give you some constraints about which airlines is interested for you to contact. There are other travel reservation agencies that play with this and they make you an offer with particularly good prices, but they do not tell you what is the airline behind it. Only if you accept the offer they will reveal this information.

Scatter/Gather

How to aggregate the responses?

- Send all (packaged into one message)
- Send one, computed using some aggregation function (e.g., average)
- Send the best one, picked with some comparison function
- Send the majority version (in case of discrepancy)

On the other direction, once we get multiple responses, how do we aggregate them? These aggregation strategies do not only contribute to enhance the scalability, but can also help to ensure the result is correct.

The simplest solution is to just package multiple responses in one reply message. A message can always carry within a collection of messages, so it's easy to do that and delegate the aggregation to the client. We can compress multiple messages with some statistics, e.g., calculating the cheapest, calculating the average, selecting the best according to some utility function.

In case of a Byzantine environment in which you do not necessarily trust all of the back ends. You can use a voting strategy. If out of five copies of the same input, one of the replies is a completely different result but the other four agree with each other, we drop the outlier and forward the majority response.

Scatter/Gather

Synchronization strategies

- When to send the aggregated response?
 - Wait for all messages
 - Wait for some messages within a certain time window
 - Wait for the first N out of M messages ($N < M$)
 - Return fastest acceptable reply
- Warning: the response-time of each component may vary and the response-time of the scatter/gather is the slowest of all component responses

In terms of how long do you wait before sending back the answer? Waiting for all the replicas to respond is bound by the slowest. Follow this strategy only if you really need to wait for all of them to guarantee complete coverage. If you expect the result will be the same, maybe you can disregard some replicas if they are too slow.

There will be a time window in which the client is expecting a response, and you will wait only until this window is about to expire before sending the best aggregated reply that has been gathered until then.

If the time window is unclear, it is possible also just to count how many replies have arrived: once the first N out of messages have arrived, you have enough responses and you can already answer the client. Sometimes you really want to make it as fast as possible, so you take the first, or the first that is acceptable. This requires to validate the answers to avoid accepting super fast replies full of random bits (or high velocity garbage).

Master/Worker

How speed up processing large amounts of input data?

split a large job into smaller independent partitions
which can be processed in parallel

The master divides the work among a pool of workers and gathers the results once they arrive

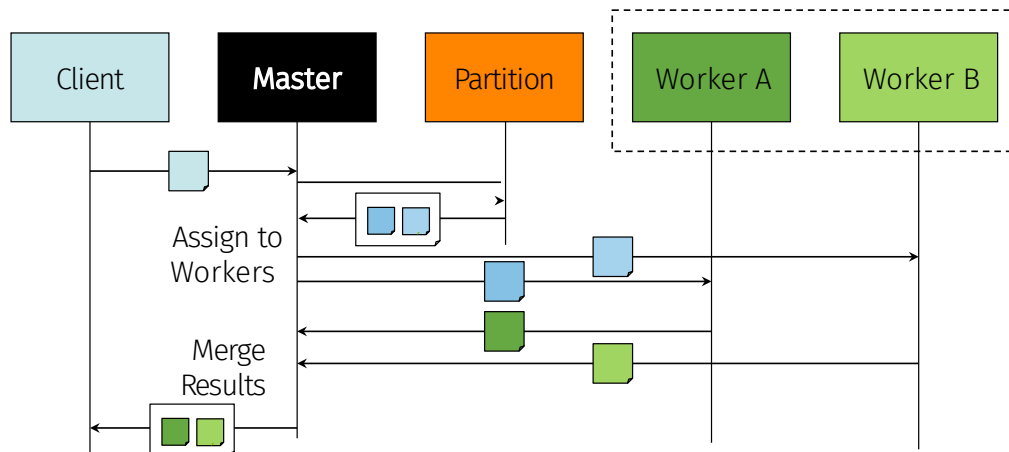
Synonyms: Master/Slave, Divide-and-Conquer

How can we refine the scatter gather pattern so that we can use it to scale, not necessarily to a large number of clients, but to deal with a single client that is however sending a large amount of input data. The idea is to split the input data into smaller partitions which are processed independently and therefore in parallel. This is the assumption that we make when applying this master worker pattern.

The master role is to accept the input data, divide it and then schedule the work among pool of identical workers. Workers are supposed to work independently and in parallel on each of the partitions. Once the results come back, then it's like in the scatter/gather pattern: we have to accumulate and concatenate the result. Once all the workers have complete we can send it back.

This pattern is also known as divide and conquer, especially when it is applied recursively. The problem is too large to be solved as a whole, so we split it and we solve each of the parts and then we have to reassemble the solution together.

Master/Worker



Example:

- Matrix Multiplication (compute each row independently)
- Movie Rendering (compute each picture frame independently)
- [Seti@home](#) (volunteer computing)

We can see the interaction has a similar structure than the scatter gather with a little but important difference: after the request that comes in, it is not directly forwarded to the workers. But before doing so, there is a partitioning step which takes the large input request and partitions it into smaller ones.

Each of these partitions needs to be assigned to a worker. While the workers perform the same computation, they don't necessarily complete it with the same performance. When all of them have processed their partition, the results are merged into a single response.

From the client perspective, such partitioning and parallel processing is also something that you can do completely transparently. The client doesn't see that its request is partitioned and then the results are reassembled back together. There is only one request coming in and one result going out. The only observation the client can make is that for some reason this implementation scales because, despite the fact that you're sending larger and larger amounts of data, the response time doesn't grow as much as one would expect. This holds if you can add more and more workers to deal with larger and larger amounts of data.

We have examples from scientific computing where this is happening quite often with parallel supercomputers. If you are trying to render a movie, for example, you know that each frame can be rendered independently, so you can actually use master/worker to scale the longer movies with higher resolution. You set up a so-called rendering farm, where you have a huge number of graphics card and processor's; you input the movie and then you don't have to actually do the computation sequentially.

If you scale this to the whole Internet, where the workers are actually running on desktops, recycling wasted computing cycles all over the world, then you have something called volunteer computing. The amount of work that you have to do is so large that you need millions of these workers all over the place. This was introduced for the search of

extraterrestrial life. Where the signals from radio telescopes were partitioned into blocks of time and also different frequencies and each block was sent out to screen savers running on desktop computers all over the world. After downloading a block, they would analyze it to search for interesting signals and possibly send back some detection events. This application became very popular, with the number of workers growing beyond the initial expectations.

Here's another scalability issue: when you really have millions of these workers coming and asking for work, you must have a huge queue of these partitions that need to be processed. Eventually they ran out of data, so they started to re-process the same data with more expensive detection algorithms.

They also came up with incentives to keep the workers interested: a reward system for people to participate and donate their CPU cycles. You could spot your name on the top 10 or the top 100 of individuals or organizations that were contributing their resources to SETI at home. Not surprisingly, when you put gamification into play, then some people are driven to win the competition. And others hacked their way to the top by running modified screensavers, which would just return some random result after downloading a piece of work without really analyzing it, to get the reward while saving on CPU usage and the corresponding energy bills.

If you do not trust the workers, how can you check that they are actually doing what they are supposed to do? What you can do is send the same partition to multiple workers, scatter, gather and compare the outcome. Or you can have some expectations about how long it is going to take to process it. If you get an answer too soon, there is something suspicious going on.

Master/Worker

Master Responsibilities

- Transparency: Clients should not know that the master delegates its task to a set of workers
- Partitioning strategies: Uniform, Adaptive (based on available worker resources), Static/Dynamic
- Fault Tolerance: if a worker fails, resend its partition to another one
- Computational Accuracy: scatter the same partition to multiple workers and compare their results to detect inaccuracies (assuming workers are deterministic)
- Master is application independent

There are many second-order design decisions that you have to make when you decide to follow the master/worker pattern. Let's discuss them more in detail.

From the master perspective, the goal is to hide from clients the fact that they are splitting the work and forwarding it to a set of parallel workers. When partitioning the work, there can be different ways to do it. If you know that workers are homogeneous – every worker has the same computational power – then you can split the work in a uniform way. If you know some workers are faster than others, you can give them more work while assigning less work to the slowest workers. This requires a model to predict how much work can every worker perform so that at the end you get back the result at the same time.

The set of workers can also fluctuate. You don't necessarily know at the time in which the partitioning happens how many workers will be available. If the computation takes a few seconds, maybe you have a reasonable expectation about how many chunks are needed, but if the computation takes a few months, it's difficult to foresee in advance the availability of workers. In this case, there can be a dynamic partitioning strategy which starts to slice the work progressively, while keeping some work from being allocated in case more workers show up. If more workers will happen to join the system, then it will either repartition, reshuffle or reassign work that was not given out yet.

We can also have a work stealing strategy when a fast worker looks for work that has been assigned to other workers that are slower.

There is also a little problem of failures, so if workers fail, we need to be aware of that because before we can send back the response to the client, we need to gather the results from all the partitions. It's important that all the workers successfully completed their job. In case of failure we have to resend the partition to be processed somewhere else and whenever we use a retry strategy to deal with failures – as we're going to see in the reliability lecture – the assumption is that it's possible to repeat the work. That

processing a partition does not have any side effects.

With deterministic computations the same result should be obtained no matter which worker produces it. In this case, it is possible to take the same input, the same partition, and have it processed by multiple workers. This way we can compare the results to detect whether some of the workers are cheating by sending back some incorrect results. The more replicas you make of each partition, the more expensive it becomes. There will be a tradeoff between how much fault tolerance you want to have and how much performance you want to have and how many replicas you want to use for processing each partition.

All of these design decisions that you make about partitioning, about retrying failed jobs, about sending out multiple copies of the same partition are design decisions that are typically independent of what the processing is about. As long as it is possible to partition the input and merge the results back together, the master does not care about its content. The part of the system that is of course application dependent is the worker. The worker needs to know how to process the input and what the application domain is about.

Master/Worker

Worker Responsibilities

- Each worker runs its own parallel thread of control and may be **distributed** across the network
- Worker **churn**: they may join and leave the system at any time (may even fail)
- Workers do not usually exchange any information among themselves
- Workers should be independent from the algorithm used to partition the work
- Workers are application domain-specific

To speed up the work by scaling over more resources, or to keep the response time flat with a larger input, you need to parallelize the work: each worker should be distributed across different cores or cluster nodes.

How stable are your workers? Here we see again, the term churn coming up. Can you assume that once you get the request from the client, you have a set of workers and they're exactly the same throughout the whole computation? Or will you have an unpredictable set of workers that come and go anytime? What if some disappear and never send you back a result?

Another assumption that simplifies applying the master/worker pattern is that when you give a processing job to a worker, the worker can perform the computation on its own. It doesn't need to know who the workers are. It doesn't have to exchange any information with them. The only communication flow is from the master to the worker and back. But workers don't have to talk to each other. There can be more complex, less embarrassing, schemes in which workers periodically need to exchange some information. But that makes running the parallel computation much more problematic.

Finally, the partitioning should be exclusive responsibility of the master. Workers should just know how to process one piece and they don't care about how the partitioning works. Unless the pattern is applied recursively, with divide and conquer, workers will decide whether it's worth to further split their partition and delegate themselves to some sub-workers the parallel processing.

Load Balancing

How to speed up processing multiple requests of many clients?

deploy many replicated instances of stateless components on multiple machines

The Load Balancer routes requests among a pool of workers, which answer directly to the clients

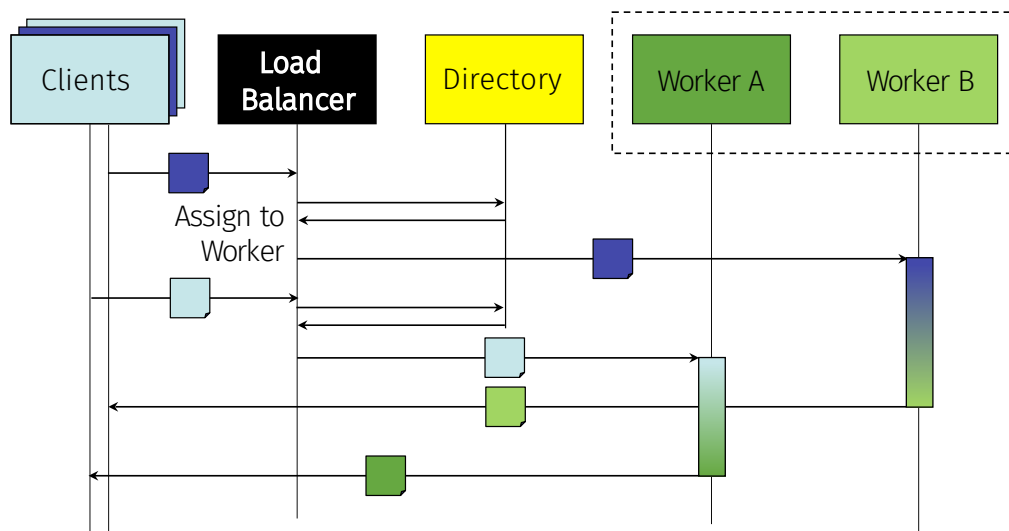
Let's see now how do we design a scalable architecture which instead of supporting one client sending a huge amount of data, it can grow to handle multiple requests of many clients. This was the original challenge of building a scalable architecture in terms of the workload.

You have one component that is the bottleneck. You can tell by measuring its response time when you have too many clients that are sending requests and they need to wait longer and longer for the responses.

If we can assume that the bottleneck component is stateless, if the responses to the requests only depend on the input and don't depend on previous requests, we can deploy many copies, many replicas of the component on different machines, and we can spread the request across all of them using a special component called the load balancer.

The load balancer is involved in routing the requests. The responses can go directly to the clients. As opposed to scatter/gather, there is no need to aggregate responses because for each request going to be processed by a worker, the response can go back directly as soon as it's ready.

Load Balancing



This is how the flow works: we have lots of clients. Here's the request from one client. The load balancer needs to find a suitable worker for processing this request. Here we have a directory that is managing the knowledge about which workers are available and where to find them. The load balancer will use the directory to perform the matchmaking between what the client wants to do and where is a worker that can do it.

Once the assignment has been made, the request will be forwarded to the worker. The worker will take it, process it, and send back the response directly.

Also in this case the client doesn't need to know where the worker is, because that is hidden behind a load balancer that is going to make the decision for the client where to send the message. The worker should have an address and where to send back the response. If you have headers in the messages, the request message will have a "reply to" header which contains the identity of the client that is supposed to receive the response.

When the next client will come in, the directory will make a different decision because this time the first worker is busy with the first client request. The decision is about: does it make sense to queue a request for this worker? What if we have another one that is free? In this case, the request will be forwarded to the worker that is free. And the worker would process it and the response will come back to the client.

As you can see, there is a scatter phase, during which different requests go to different places, but there is no gather phase at the end, because the responses go back directly.

This implementation with the directory is also dynamic, so the set of workers can change. Over time we can add more workers when there is increasing traffic and we can remove them when there is not so much traffic.

The function of the load balancer can be implemented also the lower level of the network, since it is about routing and forwarding packets to the worker endpoint addresses, which are set by the directory.

We can split the mechanism of forwarding the work, which is something that has to happen really fast – otherwise the load balancer becomes a bottleneck itself – and the directory which is the place where the strategy can be implemented to decide: how to rotate the work between different available workers? how to keep track of them? how to detect if a worker is busy or not? What if a worker fails?

The directory takes care of this, helping to keep the load balancer simple: it takes an incoming message, picks the next available worker, and forwards the message to it. These operations should be performed rather quickly. If you really want to scale, you perform them using hardware, like when introducing a specialized networking device dedicated to spraying packets around your cluster.

Load Balancing

Strategies

- Round Robin
- Random
- Random Robin
- First Available
- Nearest

Location

- Server (transparent from client)
- Client (aware of choice between alternative workers)
- Directory (e.g., DNS)

Layer

- Hardware (network device)
- Software (OS, or user-level)

There can be different strategies for balancing the work among different workers.

The simplest one is to use a counter. You know how many workers there are and for every request you increase the counter and use its value to select the worker. If you run out of workers, you start again from zero. We call this the round Robin strategy. It's super fast, but doesn't take into account any information about the availability or the current load of the workers. The assumption is that by the time you go around, the worker will be free again.

Depending on how long it takes to process each request, this assumption may be true or not. What if instead of incrementing the counter, one can skip the counter and assign requests to a random worker? As long as you pick random workers with uniform probability, we can say that statistically the load will be distributed evenly. But we don't necessarily do it always in the same order. And there can be a benefit in doing that.

You can also combine the two strategies. So you can go around once. And then shuffle the workers for the next round. And when you assign the work to all of them, then you reshuffle them for the next round.

If there is a way to keep track of the availability of the workers, it is possible to create a feedback loop between the worker that takes the work and the directory. Workers have to notify the directory about their availability state. And the directory just keeps a list of available workers and takes the first one off the list.

So there are feed-forward and feed-back strategies. The former just keep a list of work-

ers and they go through the list in some sequence. The latter assume that the worker inform the directory about their state of availability.

The nearest strategy can be used when scaling to “Web scale”: if you have clients all over the world, it can make sense to route requests to workers which are located near where the clients are. Do you know where the client request is coming from? You can forward it to a data center which is located in the same region of where the client is located so that you can minimize the latency and the ping between the worker and the client, especially when you want to send the answer back.

Sometimes this is done at the DNS level, so when you do a DNS look up, the DNS server can see where the look up is coming from and can reply with a different numeric IP address depending on where the client is located.

This brings us to the awareness about the location of the workers. How transparent do we want this to be? The idea is that the client can send a request to a certain server. The client knows the location of the server (he has to discover it using some directory). But, behind the server, the fact that the request will be load balanced is kept transparent from the client. So the load balancer knows where to find the workers, but the client doesn't. From the client perspective there is only one single endpoint. There is only one place where you talk to and behind the scenes we will do the load balancing. This leaves a bottleneck intrinsically, because of the single endpoint that all the clients have to talk to. Which implies that the load balancer becomes a hot spot since every client will have to send a message with this particular IP address even if behind it there will be others that can take care of servicing the requests. But this first step will eventually limit how much we can scale.

To avoid this, we can push the work out to the clients and this is one of the basic scaling strategy that we discussed last week. If you cannot handle the work yourself. Just get whoever is asking you to do the work to do it for you. In this case, we can give a list of endpoints to the clients. And if a client notices that one endpoints is becoming slow, then you can switch and send requests to another one. And clients can follow themselves a random strategy: out of these possible addresses, I will pick one randomly and over millions of clients that are trying to do the same, statistically the work will even out so that all the endpoints will be loaded in a similar way.

The downside is that the client has to be aware of the fact that there are different workers and you have to not only implement a client that sends a request, but you also have to implement a decentralized load balancing strategy inside the client. And you have to keep the list of workers up to date, because if you even if you choose one randomly and your choice and ends up in an obsolete address and then, then you have a problem. But if you remember about caching and the directory, the idea was that the client looks up the list in the in the directory. Keeps in the cache and only when it expires it can repeat the look up again.

If we have a load balancer that uses a directory behind the server, the client sends a request to the load balancer and the directory is located behind. However, we could also expose the directory to the client, so the client can do the look up and then send directly to the worker without all having to go through this single point.

As an alternative, we can have the directory perform the load balancing. We can have a load balancing DNS, for example. Client need to look it up anyway, because they don't know the server is, so what's the point of asking the directory: give me the location of a server? And then I will call the server and the server will have to load balance my request of our pool of workers. You might as well have the directory send back the location of the worker because the client will do the look up anyway.

If you want to build a highly scalable system, you will do a combination of these. You will start by having a load-balancing DNS. The DNS will send addresses to clients that

are, for example, nearest to them on the network. The clients will contact one of these addresses – maybe they get multiple lookup results, maybe they get only one. They will choose randomly on one of them. And once they get to these addresses, the server again will have a pool of workers behind. And will do the load balancing, so these are not necessary alternatives. But they can also be introduced in combination. When you use them in combination, you're talking about serious investment in terms of components dedicated to scalability, which can also be an overkill for more scenarios, but if you really need to go all the way, you know that you have a lot of tools at your disposal.

The load balancer can become a bottleneck for the reason that we discussed so typically also another way to mitigate this can be too implemented in hardware. There are network devices that are just doing this. They don't even open the packets, they just rewrite the addresses in the packet headers.

You can also do it at the OS level. You can have also have load-balancing Web server proxies, or you can also always implement load balancing inside your user applications.

Load Balancing

Variants

- **Stateless:** every request from any client goes to any worker (which must be stateless)
- **Session-based:** requests from the same client always go to the same worker (which could be stateful)
- **Elastic:** the pool of workers is dynamically resized based on the amount of traffic

Load balancing as we have introduced it makes the assumption that the components that you are replicating are stateless. And this is the simplest way that you can do load balancing if every request is independent. If you have a client that sends multiple requests, you don't have to remember your decision about where to send each of them. You can just take every request, send it to the next available worker and forget about it.

If however, we have stateful components, which means that the order of the requests is important and the sequence in which they are processed is important, you cannot process a request without having seen the previous ones. This puts a constraint on the load balancer: because it has to remember the previous requests from a given client so that they are sent to the same worker. The worker is accumulating state from the stream of requests which cannot be processed unless they all arrive in the right order at the worker.

We can relax the assumption that we were making before. Load balancing is easy to introduce as a scalability technique, if you assume that components are stateless. If they're stateful, you have this constraint and you solve it by establishing a session between the load balancer and the client. The load balancer, in other words, has to distinguish the

identity of the client and remember for each client which worker was assigned. Unknown clients can be assigned a worker using the previously mentioned strategies. But follow up requests from the same client must be routed to the same worker.

This also limits how flexible and how much churn, how much change you can expect from your pool of workers. If you have a stateless load balancer, there is no problem if you have seen a worker before and you don't care if the worker is new or if a worker disappears. The load balancer is completely free to assign any request to any worker.

If you use a session-based load balancer, you need to be careful that new workers only take request of new clients once a client has already sent you request before then he has to go to a worker that has seen the client before.

Another important design decision is: how long do you keep this information? How long are your sessions? If you keep them too short, then your clients will complain because they will start to get incorrect results. And if you keep them too long, you may over-constraint the life cycle of the workers which need to stay available for a long time. There is also an overhead in terms of how much session information do you need to keep track of. Since we are talking about scalability, you talk about millions of clients. It is expensive to remember all of them within the mapping table connecting each client with each worker.

The complexity of the of the protocol between the client and the load balancer also increases. There needs to be a mechanism to establish the session, but also a way to stop the session. Also after a session expires, the client will need to discover this. There could also be a way for client to prolong the session or negotiate its duration.

The last variant of load balancing is about the relationship between the pool of workers and the amount of traffic. Are you in a static environment where you have a limit on the workers that is fixed? So the capacity of your system is fixed and then just have to use them as efficiently as possible. Or is it possible when you have a traffic spike to increase the size of the pool of workers? While at one point in time you need additional capacity if the traffic decreases then you can free up some resources to do something else when the spike goes away.

Traditional load balancing assumes a static environment in which you have a set of workers and you spread the work between them using the strategies that we have discussed before and the constraint on whether the workers are stateless or stateful. If you have an elastic load balancer, things are even more complicated: the load balancer is going to measure the amount of traffic and make decisions on whether to dynamically allocate more workers or dynamically stop some of the workers.

More in detail, we have a queue of pending client requests. The load balancer consumes them by assigning them to a worker. And if the queue gets above a certain threshold, you can think that the currently available workers are not enough. You don't have enough capacity, so you might as well look for additional workers and try to increase the capacity of the system so the queue of request goes under the threshold again. If the queue gets empty, you will notice that also workers are not doing anything. Idle workers can self-destruct if they don't receive work within a given time, or they can be stopped by the load balancer and restarted only as the next wave starts to rise again.

Sharding

How to scale beyond the capacity of a single database?

partition the data across multiple databases

Route the queries to the corresponding data partition ensuring each partition remains independent and balanced

The last pattern focuses on how to scale beyond the limits of a single storage element, for example, a single database. When we talk about the capacity of a single database, this can be limited by the capacity of the physical storage. For example, going beyond 10s of terabytes. How to scale to storing petabytes of data when there is a clear physical limit on how much each storage device can provide space for?

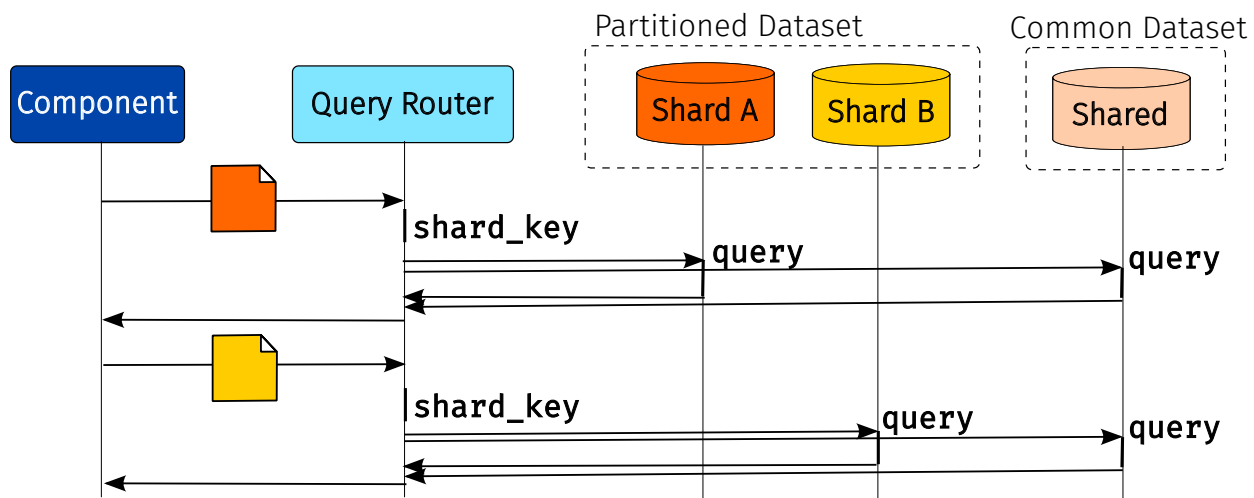
To go beyond, it should be possible to partition the data into pieces. Partition it and store it across multiple databases, so this is clearly a scale out strategy. You have been scaling up until you reach the limit, so there is no disk that can store more than 10 terabytes. Just to say a number. You can take two discs, put them side by side and now we have a storage for 20. However, we will need to partition the data so that each part fits in within the limit of the amount of storage space that we have available.

Once we partition the data then we have to know and remember where it is located. So we again we have a directory problem. We have a discovery problem. Where do we find the data? Where do we find the partition to which we can route the queries?

When a client needs to read the information or update the information we need to send the query to the corresponding partition. The challenge is to make sure that most queries get routed to only one partition. If all queries are routed everywhere, then all partitions of the whole system will be busy to service all the queries, thus limiting its scalability in terms of how many queries it can process.

While the main goal of sharding is to scale out beyond the limits of individual storage element, an additional goal is to also take advantage of the partitions to do load balancing. Different queries can be serviced in parallel by reading or writing to different partitions, which will be stored on different disks, thus increasing the amount of I/O operations which can be performed by the storage.

Sharding



Queries are directed to the corresponding shard (partition)

Queries should not involve more than one independent shard, even if they may use some shared non-sharded dataset that may need to be replicated on each shard

Sharding is also a relatively complex pattern, which can be understood as it combines together all of the ideas on how to scale that we have discussed so far. We have a query, we have to find where is the data that needs to be read or written by the query. This is discovery problem.

We have lots of queries: we have to balance the load among the partitions. We need to do so without creating hot spots, without resulting in a disk that is always going to be involved for all queries. If you can keep the traffic balanced, each storage resources will be uniformly busy and fill up at a similar rate.

Sharding works by load balancing each query coming into your storage system. There is an element that is going to make the decision on where to send the query and this will be done based on some property of the data that you're trying to access. Information extracted from the query can be used to compute the location of the target partition. We call this the key of the shard.

Shard is a synonym for partition and the key identifies which shard are you going to use. So based on the shard key, we're executing the query on the corresponding shard. The result is sent back to the original component. If a different query comes in, depending on the corresponding shard key, it may end up in a different shard. This is a way to scale the bandwidth for reading or writing into physical disks. If you have multiple partitions stored on separate disks, the amount of data which can be transferred grows if read and write operations are performed in parallel on the various disks.

What if each query reads data from a specific partition but also needs to access common data from a shared partition? If this shared data is read-only it can be replicated on each partition to ensure the queries can still be processed independently.

Sharding

- Two goals:
 - scale capacity (storage size, bandwidth, concurrent transactions)
 - balance query workload (avoid hotspots)
- What to shard?
 - Large data collections (they do not fit)
 - Hot spots (increase throughput of queries targeting a subset of the data)

When we talk about sharding, we have two goals. The first is to go beyond the limits of the available amount of storage space, the second is to grow the I/O bandwidth. How many concurrent transactions a database can process? When we hit the limit, we replicate and partition the data accordingly.

The second challenge is to how to balance the queries so that we avoid sending all the queries to the same partition. We use sharding to work with very large amounts of data which do not fit in a single storage element. To do so, we have to partition it. In case we detect a hot spot, we can use sharding to replicate it and thus increase the throughput and raise the capacity of the database to process concurrent queries.

If there is a limit on how many read transactions your database can perform, you can always make a copy of the data (it doesn't change anyway) and then you can double the throughput that you can achieve because you can read from each copy independently.

Sharding

Computing the Shard Key

- Key Ranges
- Geo-spatial (country shards)
- Time Range (year, month, day)
- Hash Partitioning
- Modulo (Number of Shards)

Usually assumes a fixed and static number of shards

The key decision that you need to make when you decide to introduce sharding is: how are you going to compute the key that you use to decide where to place the different partitions? And you already know that in your database you have keys that identify the elements that you are storing. And so one easy solution is to take the values that you assigned to your keys and map the range of values to fit within the number of partitions you plan to use.

For example, if you have to use integer keys that go from zero to 1,000,000. Then you can make 2 shards. One will be for entries from zero to 500,000 and the next shard will store the ones above 500,000. This also gives you an easy way to figure out how to transform the key into the shard identifier.

If you are storing geographical data, you can, for example, partition your data set into different countries. Since some countries are much bigger than others, you may not end up with uniform partitions.

If you work with time series, you have data that has historical perspective, going back many years. You could consider storing each year in a different shard. Given the time, it's easy to determine where to find the corresponding data. The current year is the place where you read and write, while past years go in a read-only database, since old data should no longer change.

The granularity of the partitioning is critical. If you shrink the time range and make a shard per month, week, or even day, it will be very efficient to access information on a daily basis, but if you have to run a query to aggregate the quarterly results, you will need to scatter the query and gather the results from many different partitions.

Based on the level of aggregation of your system data access patterns, you can decide how to partition it. You have to balance being too fine grained and risking that then you have to aggregate information for multiple shards or keeping it not so fine grained and then having hot spots.

When everything else fails, you can always try hashing. Take a piece of data, you hash it, and the hash can be mapped to the shard key. If hashing is expensive, you can take the modulo to map some numeric identifier within the number of shards. For example, if you have 10 shards you will take the last digit of the numeric identifier to know where

to store each object.

No matter which strategy you choose, the assumption is that the set of shards is fixed and static. So sharding is one of those decision that you make up front. How you plan to partition the data will constrain how you can access it efficiently (and vice versa). Since sharding affects the physical storage location of the information, if you change your decision, you will have to physically move data around and this is super expensive for large amounts of data.

Sharding

Looking up the Shard Key

- Business domain-driven (customer/tenant id)
- Helps to keep data balanced
- Requires to maintain a master index (i.e., a directory for shards, a ZooKeeper)

Computing the shard in general requires to come up with a function which maps your data to the shard key. The function should be deterministic, if you want to be able to consistently find where the data was placed. The advantage of computing the shard key is that you can derive the shard key from the data itself. The disadvantage is that it may not be trivial to come up with such a function.

Mapping data objects or tuples to partitions can also be seen as a lookup problem. If the previous strategies don't work out well, you can always store the key. Somewhere in your data model there will be a column which will store the address of the shard in which the data is located.

The advantage is that if you notice that some shards are getting bigger than others, you can just change the key to move data around and you don't have to change the way that you compute the key because the key is just another attribute that you associated with your data.

However, if you follow this strategy, then you have to remember where are the shards located. In other words, how do you go from the shard key to the actual location of the data? A directory can help. Given a piece of data which contains a shard key, you have a logical identifier which can be used to lookup the physical location of the corresponding shard, the address of the database server in which the shard is found.

It is important to keep this indirection, because if you store the physical location of the database and you ever need to move it, every single piece of data referring to another shard in your system will need to be updated, which again you may want to avoid. If you just remember the logical identity of the Shard, then you can just change the lookup table and you will know where to go and look for the corresponding storage element.

Re-sharding your data is expensive and typically involves down time, because while you're shuffling the data around, you really don't want to change it, and depending on how much data you're talking about, this takes time.

Sharding

- Changing the number of shards or the sharding strategy may require an expensive and complex repartitioning operation
- Transactions should only involve one shard (some shared data may need to be replicated on each shard)
- Sharding was originally implemented outside the data layer, sometime as part of the application logic. Some databases are starting to offer native sharding support

Sharding was introduced when people were hitting the limits of how much a single database could manage, and if you have such limit in a component you keep, you keep the component as it is. You make copies of it and in front of it you develop something that can give you a unified view over all of these copies.

Sharding was originally implemented by the applications accessing the database. Over time, this feature, since it was used in many large-scale applications, migrated down into the databases themselves. Nowadays when you buy a database that can scale beyond a certain size limit, it will offer you some native sharding support and the advantages that from the application side, it looks like the database can indeed store petabytes of data. Behind the scenes there will be all of these strategies available to be configured which can help you to scale.

Sharding

Different systems use different terms to name data partitioning for scalability

Shard	MongoDB, Elasticsearch, SolrCloud
Region	HBase
Tablet	Bigtable
vnode	Cassandra, Riak
vbucket	Couchbase

References

- Martin L. Abbott, Michael T. Fisher, The Art of Scalability, Pearson, 2015
- Gregor Hohpe and Bobby Woolf, [Enterprise Integration Patterns](#), Addison-Wesley, October 2003, ISBN 0321200683
- Martin Kleppmann, Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable and Maintainable Systems, O' Reilly, 2017, ISBN 978-1-449-37332-0

Software Architecture

Availability and Services

11

Contents

- Components vs. Services
- Monitoring Availability
- Impact of Downtime
- Control Cascading Failures: Circuit Breaker, Canary Call
- Redundancy with Replication: Consistency vs. Availability
- Event Sourcing